

TrainWise

Mobile App | Complete Project Reference

React Native + Expo / ASP.NET Core 8 / SQL Server

Every screen. Every endpoint. Every table.

Use this document as the canonical reference for any future Claude / Claude Code conversation on this project.

Author: Liron Vaknin

Generated: 2026-04-16

Document built by Claude Code (claude-opus-4-6)

Table of Contents

1. Project Overview
2. Technology Stack
3. Repository Layout
4. Architecture (4 layers)
5. SQL Server Database
 - 5.1 All tables
 - 5.2 All stored procedures
 - 5.3 AC-Ratio training-load algorithm
6. Backend - C# ASP.NET Core 8
 - 6.1 Program.cs / appsettings
 - 6.2 Controllers (12)
 - 6.3 Business Logic (BL)
 - 6.4 Data Access Layer (DAL)
 - 6.5 Models
7. Frontend - React Native Expo
 - 7.1 App bootstrap
 - 7.2 AuthContext + session flow
 - 7.3 Navigation tree
 - 7.4 api.js / services/api.js (split)
 - 7.5 Health Connect integration
 - 7.6 Sync pipeline
 - 7.7 Theme & components
 - 7.8 Every screen
8. End-to-End Data Flows
9. Build & Run instructions
10. Known Issues & Architectural Gaps
11. File Index

1. Project Overview

TrainWise is a mobile fitness application that helps amateur and semi-professional athletes monitor training load and prevent injuries. Users log workouts (manually or imported from Health Connect on Android), and the system calculates an Acute:Chronic workload ratio (AC ratio), classifies risk on a Green / Yellow / Red traffic-light scale, and surfaces personalized recommendations. Coaches can connect to their trainees through a QR-code based handshake and receive a summary of each trainee's current load.

Primary user stories

- As an athlete, I log a workout and immediately see whether my training load is in a safe zone or trending towards overload.
- As an athlete, I get personalized recommendations after each session (rest, light activity, push harder).
- As an athlete, I report an injury so the system tightens my load thresholds for the recovery window.
- As a coach, I scan a trainee's QR code, see all my trainees on one screen with their current AC ratio and load level.
- As a user, I import workouts from Google / Samsung Health (Android Health Connect) so I do not

have to enter them manually.

Project authors & branches

Author: Liron Vaknin

Repo: TrainWise_GitHub (single git repo at TrainWise_Project root)

Backend branch: main / Lirone's-Branch (mirror of TrainWise_GitHub)

Recent commits: 793af4b 'Adding .sln File', 0626afb 'Server_Side'

2. Technology Stack

Frontend

- React Native 0.81.5 + React 19.1.0
- Expo SDK 54 (managed workflow)
- React Navigation v7 - native stack + bottom tabs
- axios 1.x for HTTP
- @react-native-async-storage/async-storage - session persistence
- react-native-chart-kit - weekly load bar charts
- react-native-qr-code-svg - coach/trainee handshake
- react-native-health-connect - Android Health Connect (real device only)
- @expo/vector-icons (Ionicons) - tab bar icons

Backend

- ASP.NET Core 8.0 (net8.0) Web API
- Microsoft.Data.SqlClient 6.1.3 - SQL Server access
- Swashbuckle.AspNetCore 6.6.2 - Swagger / OpenAPI
- Built-in Microsoft.Extensions.Configuration for appsettings.json
- CORS: AllowAnyOrigin (development), no auth middleware

Database

- Microsoft SQL Server Express
- Instance: Lirone\SQLEXPRESS - database: TrainWise
- Connection: Integrated Security (Windows auth), Encrypt=False
- 16 tables, 49 stored procedures (the entire DAL layer is sproc-based)

Mobile testing path

- Android phone -> Expo Go app -> connects to PC via USB
- adb reverse tcp:5249 tcp:5249 - exposes PC port 5249 on phone
- Frontend uses base URL http://127.0.0.1:5249 - resolved on PHONE
- Backend runs HTTP only (HTTPS dev cert is not trusted by phone)

3. Repository Layout

```

TrainWise_Project/
  TrainWise_GitHub/
    BL/
      Models/
      Controllers/
      DAL/
        DBservice.cs
        Program.cs
        appsettings.json
        TrainWise.csproj
        TrainWise.sln
    <-- C# backend (active)
    -- Business logic (12 classes)
    -- POCOs (15 classes)
    -- ASP.NET controllers (12)
    -- Data access (14 classes)
    -- Base: SqlConnection + sproc helper
    -- ASP.NET startup
    -- ConnectionString to SQL Express
    -- Net8 project file
    -- Solution file

  TrainWiseExpo/
    App.js
    index.js
    app.json
    package.json
    assets/images/wowwow.png
    src/
      api/
        api.js
        AuthContext.js
        HealthConnectService.js
        HealthConnectService.mock.js
        SyncService.js
        useSyncWorkouts.js
        GoogleFitScreen.js
      services/
        api.js
      navigation/
        NavigationStack.js
      screens/
      components/
      theme/colors.js
      constants/google.js
    <-- React Native frontend (active)
    -- Provider chain root
    -- registerRootComponent(App)
    -- Expo config
    -- RN 0.81 / React 19 / Expo 54
    -- Brand logo (cover screen)
    -- AuthContext + axios + Health Connect
    -- Base axios + auth/login + ActivityLog
    -- Global session (user, login, logout)
    -- Real native module wrapper
    -- Expo Go fallback (no native)
    -- HC -> backend orchestration
    -- Hook: triggerSync(), state
    -- Health Connect UI screen
    -- The other axios with all CRUD funcs
    -- AuthStack / AppStack tabs
    -- 11 screens
    -- Card, ScreenHeader, PrimaryButton
    -- Color/Font/Spacing tokens
    -- GOOGLE_WEB_CLIENT_ID

  github - server-side/
    <-- Old duplicate (BL/Controllers/DAL
    only, no .csproj). Candidate for
    deletion - awaiting confirmation.

  generate_docs_pdf.py
  TrainWise_Project_Documentation.pdf
  <-- This script

  ../SQL/TrainWiseDB_Script.sql
  <-- DB schema + seed (UTF-16LE!)

```

4. Architecture - 4 layers

Every request flows through exactly four layers. Each layer talks only to the one directly below it. The DAL never instantiates a BL, BL never reads HttpContext, etc. This keeps the boundaries clean.

```

Mobile UI (React Native screens)
  | axios (HTTP/JSON)
  v
Controller      -- thin: validate route, call BL, map exceptions
  |
  v
BL (Business Logic) -- input validation, cross-entity rules
  |
  v
DAL (Data Access)  -- inherits DBservice, calls stored procedure
  |
  v
SQL Server        -- stored procedures own the algorithm

```

Why stored procedures?

All business-critical reads (GetActivityLogsForLoad, GetUserLoadContext) and writes (SaveDailyLoad) live in T-SQL. The C# DAL is a thin marshaller: open connection, call sproc, map rows. The AC-Ratio formula itself is computed in the DailyLoadDAL (C# pulls the activity rows, runs the math, then calls sp_SaveDailyLoad to persist).

5. SQL Server Database

5.1 Tables (16)

Users

UserID (PK identity), FullName, BirthYear, Gender, Height, Weight, ActivityLevel, CreatedAt, DeviceType, UserName, Email, Password char(8) (plain text!), ExperienceLevel (1=Beginner, 2=Regular, 3=Advanced), BaseLineDailyLoad, BaseLineWeeklyLoad, HealthDeclaration bit, ConfirmTerms bit, TermConfirmationDate, ProfileImagePath, IsBaselineEstablished bit, BaselineEstablishedDate, IsCoach bit

ActivityTypes

ActivityTypeID (PK identity), TypeName, IntensityFactor decimal(5,2). Seeded with Running 1.30, Walking 0.80, Cycling 1.10, CrossFit 1.50, Swimming 1.20.

ActivityLogs

ActivityID (PK), UserID, ActivityTypeID, StartTime, EndTime, DistanceKM, AvgHeartRate, MaxHeartRate, CaloriesBurned, SourceDevice, ExertionLevel tinyint (1-10), Duration smallint (minutes), CalculatedLoadForSession smallint (Duration x Exertion x IntensityFactor), IsConfirmed bit.

DailyLoad

LoadID (PK), UserID, Date, AcuteLoad float (last 7 days), ChronicLoad float (last 28 days), AC_Ratio float (acute / chronic), StressScore int (0-100), LoadLevel nvarchar(20) ('Green' / 'Yellow' / 'Red').

LoadParameters

Single-row config table. BeginnerDailyLoad / RegularDailyLoad / AdvanceDailyLoad (smallint), BeginnerAcuteLoad / RegularAcuteLoad / AdvanceAcuteLoad, LowLoadRatio (float), SafeZoneLowRange (float), SafeZoneHighRange (float), OverLoad (float). Reference values used until a personal baseline is established.

Recommendations

RecID (PK), UserID, Date, LoadLevel, RecommendationText nvarchar(max), Type. System-generated suggestions per daily load calculation.

Coaches

CoachID (PK), FullName, Email, UserID (nullable link to Users).

CoachTrainees

Composite PK (CoachID, UserID), ConnectionDate, AllowNotifications. Many-to-many: a coach can have many trainees.

CoachRecommendations

RecID (PK), CoachID, UserID, Date, Title, Text. Coach-authored messages to a specific trainee.

InjuryTypes

InjuryTypeID (PK), InjuryName. Catalog of injury labels (Knee Pain, Shin Splints, Ankle Sprain, ...).

InjuryCategories

Composite PK (InjuryTypeID, CategoryName). Many categories per injury type.

InjuriesReports

InjuryID (PK), UserID, InjuryTypeID, Date, Severity (1-10 in UI but BL validates 1-5), Notes nvarchar(max), IsActiveInjury bit.

TrainingGoals

GoalID (PK), GoalName. Catalog of goals user can pick from.

UserTrainingGoals

Composite PK (UserID, GoalID). Many-to-many user <-> goal.

UserActivityPreferences

Composite PK (UserID, ActivityTypeID). Activities the user marks as preferred.

UserDevices

DeviceID (PK), UserID, DeviceName, LastSync datetime, PermissionsGranted bit. Used by SyncService.updateDeviceLastSync to track the last successful Health Connect pull.

5.2 Stored Procedures (49)

The DAL never builds dynamic SQL; every call goes through one of these. Naming convention: sp_VerbNoun.

Users

- sp_InsertUser
- sp_UpdateUser
- sp_DeleteUser
- sp_GetUserById
- sp_GetAllUsers
- sp_GetUserSummary
- sp_GetUserLoadContext
- sp_UpdateUserBaseline
- sp_UpdateUserProfileImage
- sp_LoginUser (email + 8-char plaintext password)

Activity

- sp_InsertActivityLog
- sp_UpdateActivityLog
- sp_DeleteActivityLog
- sp_GetActivityLogsByUser
- sp_GetActivityLogsForLoad (last 28 days, with IntensityFactor joined - feeds C# AC ratio calc)
- sp_GetAllActivityTypes
- sp_InsertActivityType

Daily load

- sp_GetDailyLoadByUser
- sp_SaveDailyLoad (upsert + optional baseline establishment)

Recommendations

- sp_InsertRecommendation
- sp_GetRecommendationsByUser
- sp_InsertCoachRecommendation
- sp_GetCoachRecommendationsByUser

Coach

- sp_InsertCoach
- sp_DeleteCoach
- sp_GetCoachById
- sp_GetCoachByUserId
- sp_ConnectTraineeToCoach
- sp_DisconnectTrainee
- sp_GetTraineesWithLoadByCoach
- sp_GetTraineeDailyLoadForCoach

Injuries

- sp_InsertInjuryReport
- sp_GetInjuriesByUser
- sp_GetActiveInjuriesByUser
- sp_GetAllInjuryTypes

Goals & Preferences

- sp_AddUserTrainingGoal
- sp_RemoveUserTrainingGoal
- sp_AddUserActivityPreference
- sp_RemoveUserActivityPreference
- sp_GetAllTrainingGoals

Devices & config

- sp_InsertUserDevice
- sp_UpdateUserDevice
- sp_GetUserDevices
- sp_GetLoadParameters

5.3 AC-Ratio Training-Load Algorithm

TrainWise classifies each day on a Green / Yellow / Red traffic light. The classification is based on the Acute:Chronic Workload Ratio (AC ratio), a published sports-medicine model.

Per-session load

```
sessionLoad = Duration (min) x Exertion (1-10) x IntensityFactor
```

IntensityFactor comes from ActivityTypes:

```
Running 1.30   Walking 0.80   Cycling 1.10
CrossFit 1.50  Swimming 1.20
```

Acute & chronic load

AcuteLoad = sum of session loads in the last 7 days

ChronicLoad = sum of session loads in the last 28 days, divided by 4

AC_Ratio = AcuteLoad / ChronicLoad (NULL when ChronicLoad = 0)

Traffic-light classification

Lookup thresholds from LoadParameters:

```
LowLoadRatio      (e.g. 0.8)
SafeZoneLowRange  (e.g. 0.8)
SafeZoneHighRange (e.g. 1.3)
OverLoad          (e.g. 1.5)
```

```
if AC_Ratio < LowLoadRatio      -> Yellow ('undertraining')
elif SafeLow <= AC_Ratio <= SafeHigh -> Green (safe zone)
elif SafeHigh < AC_Ratio < OverLoad -> Yellow (caution)
else                             -> Red (overload risk)
```

Bootstrap mode (no baseline yet)

Until the user has 28 days of data, IsBaselineEstablished = 0 and the system uses LoadParameters absolute thresholds (Beginner / Regular / Advance daily/acute) keyed off ExperienceLevel. Once the baseline window closes, sp_SaveDailyLoad upgrades the user to personal baselines (BaseLineDailyLoad / BaseLineWeeklyLoad).

Active-injury modifier

If sp_GetUserLoadContext returns HasActiveInjury=1, the C# layer tightens the Yellow/Red thresholds (currently a TODO marker in DailyLoadDAL - see Known Issues).

6. Backend - C# ASP.NET Core 8

6.1 Program.cs / appsettings

Startup is intentionally minimal:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddCors(o => o.AddDefaultPolicy(p =>
    p.AllowAnyOrigin().AllowAnyHeader().AllowAnyMethod()));
var app = builder.Build();
if (app.Environment.IsDevelopment()) {
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseHttpsRedirection(); // <-- comment out for phone testing
app.UseCors();
app.UseAuthorization(); // present but no AddAuthentication()
app.MapControllers();
app.Run();
```

appsettings.json

```
"ConnectionStrings": {
  "myProjDB": "Data Source=Lirone\\SQLEXPRESS;Initial Catalog=TrainWise;Integrated Secur
ity=True;Encrypt=False"
}
```

Listening URL

HTTP: http://localhost:5249 (the URL the phone reaches via adb reverse)

HTTPS: https://localhost:7126 (dev cert, do NOT use from phone)

Swagger: http://localhost:5249/swagger

6.2 Controllers

All controllers follow the same shape: instantiate the matching BL in the constructor, wrap each action in try / catch, return Ok / BadRequest / 500.

UserController

[ApiController] [Route("api/users")]

- POST /api/users - Create
- PUT /api/users/{id} - Update
- DELETE /api/users/{id} - Delete
- GET /api/users/{id} - GetById
- GET /api/users - GetAll
- PUT /api/users/{id}/profile-image - SetProfileImagePath

ActivityLogController

[ApiController] [Route("api/activitylog")]

- POST /api/activitylog - Create
- PUT /api/activitylog - Update
- DELETE /api/activitylog/{id} - Delete
- GET /api/activitylog/user/{userId} - GetUser

DailyLoadController

[ApiController] [Route("api/dailyload")]

- POST /api/dailyload/user/{userId}/calculate - Run AC-ratio calc for today, save, return tuple
- GET /api/dailyload/user/{userId} - GetUser (history)

RecommendationController

[ApiController] [Route("api/recommendation")]

- POST /api/recommendation - Create
- GET /api/recommendation/user/{userId} - GetByUser

InjuryReportController

[ApiController] [Route("api/injuryreport")]

- POST /api/injuryreport - Create
- GET /api/injuryreport/user/{userId} - GetByUser
- GET /api/injuryreport/user/{userId}/active - GetActive

InjuryTypesController

[ApiController] [Route("api/injurytypes")]

- GET /api/injurytypes - GetAll

CoachController

[ApiController] [Route("api/coach")]

- GET /api/coach/{id} - GetById
- GET /api/coach/{id}/trainees - GetTrainees

TrainingGoalsController

[ApiController] [Route("api/traininggoals")]

- GET /api/traininggoals - GetAll

UserGoalsController

[ApiController] [Route("api/usertraininggoals")]

- POST /api/usertraininggoals - AddGoal
- DELETE /api/usertraininggoals/{userId}/{goalId} - RemoveGoal

UserActivityPreferencesController

[ApiController] [Route("api/useractivitypreferences")]

- POST /api/useractivitypreferences - AddPreference
- DELETE /api/useractivitypreferences/{userId}/{actId} - RemovePref

CoachRecommendationsController

(MISSING [ApiController] AND [Route] attributes!)

- Effectively unreachable as written.
- Frontend tries /api/coachrecommendations/user/{id} - will 404.

UserDevicesController

(MISSING [ApiController] AND [Route] attributes!)

- Effectively unreachable as written.
- Frontend tries /api/userdevice/user/{id} - will 404.
- Also: src/api/api.js calls /api/users/{id}/devices - inconsistent.

WARNING: No AuthController exists

Frontend /api/auth/login points to a controller that does NOT exist in the codebase. The sp_LoginUser stored procedure IS present in the database, so the only missing piece is a thin AuthController + AuthBL + UserDAL.LoginByEmailPassword that wraps it. Until that lands, the LoginScreen will throw on every submit.

6.3 Business Logic (BL)

UserBL

Create / Update / Delete / GetById / GetAll / SetProfileImagePath. Validates: FullName non-empty, BirthYear in [1950..now], Height>0, Weight>0, ActivityLevel in [1..5], DeviceType non-empty.

ActivityLogBL

Create / Update / Delete / GetByUser. Validates UserID, ActivityTypeID, StartTime<EndTime, DistanceKM>=0, and that the user actually exists (calls UserDAL.GetUserById).

DailyLoadBL

CalculateForDay(userId, date) -> tuple(acute, chronic, acRatio, stressScore, loadLevel).
Validates user exists and date<=today. GetByUser(userId) -> List<DailyLoad>.

RecommendationBL

Create (validates UserID, LoadLevel, RecommendationText, Type) and GetByUser.

InjuryReportBL

Create (Severity must be 1..5, Date<=today, user must exist) and GetByUser. NOTE: frontend slider is 1..10 - mismatch.

CoachBL

GetTrainees(coachId) -> List<TraineeSummary>. GetTraineeLoad(coachId, userId) -> List<DailyLoad>.

CoachRecommendationBL

Create + GetByUser. Validates CoachID, UserID, Title, Text non-empty.

UserDeviceBL

Create / Update / GetByUser. Validates UserID and DeviceName.

InjuryTypeBL

GetAll() catalog reader.

TrainingGoalCatalogBL

GetAll() catalog reader.

UserActivityPreferenceBL

AddPreference / RemovePreference. Both validate userId>0 and activityTypeId>0.

UserTrainingGoalBL

AddGoal / RemoveGoal. Both validate userId>0 and goalId>0.

6.4 Data Access Layer (DAL)

Every DAL inherits from DBService. The base class supplies two methods:

```
protected SqlConnection Connect()
    -> reads ConnectionStrings:myProjDB from appsettings, opens conn

protected SqlCommand CreateCommandWithStoredProcedure(
    string spName, SqlConnection con, Dictionary<string,object> params)
    -> builds SqlCommand of type StoredProcedure, binds parameters
```

Method-level summary (one bullet per method):

- ActivityLogDAL.Insert / Update / Delete / GetByUser
- ActivityTypeDAL.GetAll / Insert
- CoachDAL.GetCoachById / GetTraineesWithLoad / GetTraineeLoadHistory
- CoachRecommendationDAL.Insert / GetByUser
- DailyLoadDAL.CalculateDailyLoad (the algorithm) / GetDailyLoadByUser
- InjuryReportDAL.InsertInjuryReport / GetInjuriesByUser
- InjuryTypeDAL.GetAll
- RecommendationDAL.InsertRecommendation / GetByUser
- TrainingGoalDAL.GetAll
- UserActivityPreferenceDAL.AddPreference / RemovePreference
- UserDAL.InsertUser / UpdateUser / DeleteUser / GetUserById / GetAllUsers / UpdateUserProfileImage (NO Login method yet)
- UserDeviceDAL.Insert / Update / GetByUser
- UserTrainingGoalDAL.AddGoal / RemoveGoal

DailyLoadDAL.CalculateDailyLoad - the heart of the app

The C# orchestrator: pulls activity rows via `sp_GetActivityLogsForLoad` (last 28 days, joined to ActivityTypes for IntensityFactor), pulls thresholds + baseline via `sp_GetUserLoadContext`, computes Acute / Chronic / AC ratio / stress / load level, persists via `sp_SaveDailyLoad` (which also upgrades the user from bootstrap-mode to personal-baseline mode when ready), and returns a tuple back up to the BL.

6.5 Models (POCOs in BL/Models/)

User: UserID, FullName, BirthYear, Gender, Height, Weight, ActivityLevel, CreatedAt, DeviceType, ProfileImagePath? - MISSING Email, Password, IsCoach, ExperienceLevel, BaseLine* fields that exist in the SQL table. See Known Issues.

ActivityLog: ActivityID, UserID, ActivityTypeID, StartTime, EndTime, DistanceKM, AvgHeartRate, MaxHeartRate, CaloriesBurned, SourceDevice - MISSING ExertionLevel, Duration, CalculatedLoadForSession, IsConfirmed (frontend sends them, backend silently drops).

ActivityType: ActivityTypeID, TypeName (no IntensityFactor!)

DailyLoad: LoadID, UserID, Date, AcuteLoad, ChronicLoad, AC_Ratio?, StressScore, LoadLevel

Recommendation: RecID, UserID, Date, LoadLevel, RecommendationText, Type

Coach: CoachID, FullName, Email

CoachTrainee: CoachID, UserID (just the relation)

CoachRecommendation: RecID, CoachID, UserID, Date, Title, Text

InjuryReport: InjuryID, UserID, InjuryTypeID, Date, Severity, Notes - no IsActiveInjury field

InjuryType: InjuryTypeID, InjuryName

TraineeSummary: UserID, FullName, BirthYear, Gender, DeviceType + LastDate, AcuteLoad, ChronicLoad, AC_Ratio, LoadLevel (the row rendered on a coach's trainee list)

TrainingGoal: GoalID, GoalName

UserActivityPreference: UserID, ActivityTypeID

UserDevice: DeviceID, UserID, DeviceName, LastSync, PermissionsGranted

UserTrainingGoal: UserID, GoalID

7. Frontend - React Native Expo

7.1 App bootstrap

```
index.js
  registerRootComponent(App)

App.js
  <AuthProvider>
    <SafeAreaProvider>
      <NavigationContainer>
        <AppNavigator />
      </NavigationContainer>
    </SafeAreaProvider>
  </AuthProvider>

app.json
  slug: TrainWiseExpo,  scheme: trainwiseexpo,
  newArchEnabled: true, no plugins
```

7.2 AuthContext + session flow

src/api/AuthContext.js owns the session. There are NO JWT tokens. After login the User object is normalized and persisted to AsyncStorage under '@trainwise_user'. On every app start, bootstrapAsync rehydrates from storage and flips isLoading=false.

Hook surface

```
const {
  user,           // full User object or null
  userId,         // shorthand: user?.userId
  deviceId,       // user?.deviceId (set later by sync flow)
  isLoggedIn,     // !!user
  isLoading,      // true during bootstrap & login
  error,          // last error string
  login,          // (email, password) => Promise<User>
  logout,         // () => clears AsyncStorage + state
  updateUser,     // (partial) => merge + persist
} = useAuth();
```

WARNING: Hooks rule - call inside the component, never at module scope

All five fixed screens (Add Workout, Connect QR, Injury Report, Settings, Warnings Dashboard) had `const { userId } = useAuth();` at the TOP of the file (module scope). React's useContext returned null, the app crashed with `useContext of null` on the very first render. Always put `useAuth()` INSIDE the component function body, after the function signature.

7.3 Navigation tree

```

AppNavigator (decides based on isLoggedIn)
|
+-- AuthStack (when NOT logged in)
|   +-- Welcome (WelcomeScreen)
|   +-- Login (LoginScreen)
|
+-- AppStack (BottomTabs, when logged in)
    +-- HomeTab -> HomeStack
        |   +-- HomeMain (HomeScreen)
        |   +-- Stats (StatsScreen)
        |   +-- Warnings (WarningsDashboardScreen)
        |   +-- AddWorkout (AddWorkoutScreen)
        |   +-- InjuryReport (InjuryReportScreen)
        |   +-- WorkoutSummary (WorkoutSummaryScreen)
        |   +-- Settings (SettingsScreen)
        |   +-- ConnectQR (ConnectQRScreen)
    +-- HealthTab -> HealthStack
        |   +-- HealthConnectMain (GoogleFitScreen)
    +-- ProfileTab -> ProfileStack
        +-- ProfileMain (ProfileScreen)

```

7.4 api.js (TWO files - know which is which!)

WARNING: Two parallel API surfaces

src/api/api.js and src/services/api.js are both axios clients talking to the same backend, but with different baseURL conventions and different naming. Both are imported across the app. Don't merge them lightly - migrating consumers is non-trivial.

src/api/api.js (BASE_URL = http://127.0.0.1:5249)

axios instance + login(email, password) which POSTs to /api/auth/login (THIS ENDPOINT DOES NOT EXIST yet). Plus: getActivityLogs(userId), postActivityLog, putActivityLog, deleteActivityLog, getUserDevices, postUserDevice, putUserDevice, setBaseURL, getBaseURL.

src/services/api.js (API_BASE_URL = http://127.0.0.1:5249/api)

axios instance pre-rooted at /api. Exposes: getUserById, updateUser, updateProfileImage, getAllActivityTypes, createActivityLog, getActivityLogsByUser, getDailyLoadByUser, calculateDailyLoad, getRecommendationsByUser, getCoachRecommendationsByUser, getAllInjuryTypes, createInjuryReport, getInjuriesByUser, getActiveInjuriesByUser, getCoachById, getTraineesByCoach, getAllTrainingGoals, addUserTrainingGoal, removeUserTrainingGoal, addUserActivityPreference, removeUserActivityPreference, getUserDevices, registerDevice.

Both files document the adb-reverse trick at the top: phone reaches PC localhost via USB-bridged 127.0.0.1:5249. WiFi-only setups require swapping the IP to the PC's LAN IPv4 (e.g. 192.168.1.117) and disabling AP isolation on the router.

7.5 Health Connect integration

src/api/HealthConnectService.js wraps the native react-native-health-connect module. It uses CommonJS require() instead of ES import so the load can be wrapped in a try/catch - in Expo Go the native module isn't registered and TurboModuleRegistry.getEnforcing throws. When that happens, the module falls back to HealthConnectService.mock.js which returns empty/zero values for every function. Result: app runs in Expo Go, Health Connect actually works on a real Android build.

Exercise type mapping (HC -> ActivityTypeID)

```

56 RUNNING      -> 1
79 WALKING      -> 2
 8 BIKING       -> 3
80 WEIGHTLIFTING -> 4
 * (other)      -> 5 (default)

```

Required permissions

- READ_STEPS
- READ_DISTANCE
- READ_EXERCISE
- READ_HEART_RATE
- READ_TOTAL_CALORIES_BURNED

7.6 Sync pipeline

`src/api/SyncService.syncWorkoutsToBackend(userId, deviceId, lookbackDays=7)` executes seven steps:

1. `initializeHealthConnect()` - abort if SDK status false
2. `checkPermissions()` - abort if any missing
3. `getStructuredWorkouts(startDate, endDate)` from Health Connect
4. `getActivityLogs(userId)` from backend - for dedup
5. Deduplicate: workouts whose `StartTime` is within 60s of an existing log are dropped
6. POST each new workout via `postActivityLog`
7. `updateDeviceLastSync(userId, deviceId)` - PUT new `LastSync`

Returns { success, synced, skipped, errors[], workouts[] }.

useSyncWorkouts() hook

Wraps `SyncService` for screens. Exposes `triggerSync`, `requestHCPPermissions`, `checkHCPPermissions`, plus state `isSyncing`, `lastSyncTime`, `syncResult`, `error`, `permissionsGranted`, `syncAttempts`.

7.7 Theme & components

theme/colors.js

```

Colors:
  background      #0A1628 (dark navy)
  cardBackground  #132036
  primary         #E91E63 (TrainWise pink)
  primaryDark     darker pink
  primaryLight    lighter pink
  accent          #FF6090
  textPrimary     #FFFFFF
  textSecondary   light grey
  textMuted       muted grey
  inputBackground / inputBorder / border
  green / yellow / red / shadow

Fonts: titleSize 28, subtitleSize 18, bodySize 15, captionSize 12,
       bold '700', semiBold '600'

Spacing: xs 4, sm 8, md 16, lg 24, xl 32, xxl 48

```

Components

- `Card({children, style})` - rounded card with shadow + horizontal margin
- `PrimaryButton({title, onPress, loading, disabled, style})` - filled pink button, `ActivityIndicator` while loading
- `ScreenHeader({title, subtitle, onBack})` - centered title + optional back-arrow at top-left

7.8 Screens (11)

WelcomeScreen.js (Auth)

Brand entry screen with the wowowow.png logo + CurvedText laying each character along an arc. Two CTAs: 'Sign Up' -> SignUp; 'HERE!' (under SIGN IN prompt) -> Login.

LoginScreen.js (Auth)

Themed login form (pink/purple/mint border). Inputs: email + password. Calls useAuth().login(email, password) which POSTs to /api/auth/login (AuthController -> UserBL.Authenticate). 'NEW HERE? CREATE AN ACCOUNT' link -> SignUp.

SignUpScreen.js / SignUpFinal.js (Auth)

Two-step registration. SignUpScreen collects identity + body metrics; SignUpFinal collects credentials + activity preferences. Final POST -> /api/Users (registerUser(payload) -> UsersController -> UserBL.Create). Server hardcodes ProfileImagePath, baseline fields, CreatedAt -- those keys are ignored if sent by the client.

HomeScreen.js (HomeTab)

Greeting + settings gear + avatar. Weekly bar chart driven by buildWeeklyData(backendLogs) -- BACKEND ONLY (no Health Connect fallback so a deleted log stays empty). Bar colors come from session-load thresholds: <=150 green, <300 yellow, <500 orange, 500+ red. Auto-refreshes via useFocusEffect when the user returns to the tab. Three big buttons: Add Workout, See Warnings, Report Injury.

StatsScreen.js (HomeStack)

Two view modes. Overview = weekly bar chart + 'X training sessions this week' summary. Detail = zoomed 3-day chart + edit form (duration / exertion / distance / pulse). Imports getBarColor from HomeScreen so the colors match. Selected bar keeps its load color (opacity + width changes mark selection -- NOT a fixed pink). Empty days show all zeros. Apply changes PUTs/POSTs to /api/ActivityLog and recalcs the edited day + today via calculateDailyLoad.

WarningsDashboardScreen.js (HomeStack)

Computes status, AC ratio, stress, and the weekly bar chart client-side from ActivityLogs (NOT from DailyLoad rows -- those are 7-day rolling snapshots and would leak prior-week sessions into this-week bars). For the displayed Sun-Sat: acute = sum of session loads in week, chronic = sum(prior 21 days)/3, ratio = acute/chronic. Color thresholds: ratio<0.8 Green, 0.8<=ratio<=1.3 Yellow, ratio>1.3 Red. Refresh button recalcs all 7 days (loop i=6..0) so stale DailyLoad rows are flushed.

AddWorkoutScreen.js (HomeStack)

Loads getAllActivityTypes() into chip selector (fallback list hard-coded if API fails). Inputs: duration (min), exertion (1-10 dot row), distance (km), avg + max heart rate. Computes sessionLoad = duration * exertion (intensityFactor was REMOVED across DB/backend/frontend), POSTs createActivityLog(...), then calls calculateDailyLoad(userId), navigates to WorkoutSummary with the result.

WorkoutSummaryScreen.js (HomeStack)

Read-only result page. Receives 'summary' via route.params: activityName, duration, exertion, sessionLoad, loadLevel, acuteLoad, chronicLoad, acRatio, stressScore, recommendation. Displays Session Details card, Load Assessment card with colored Green/Yellow/Red badge + 4 metrics grid (Acute Load 7-day, Chronic Load 28-day, AC Ratio, Stress 0-100), Recommendation card.

InjuryReportScreen.js (HomeStack)

Top-level screen now: loads getAllInjuryTypes(), reports a NEW injury (chip selector + severity 1-10 + notes), AND shows a drill-down card with active-injury count -> navigates to ActiveInjuriesScreen. Refreshes activeCount on focus. POSTs createInjuryReport({ userID, injuryTypeID, date, severity, notes, isActiveInjury: true }).

ActiveInjuriesScreen.js (HomeStack)

Lists every active injury for the user (hosted as a separate screen so the report form stays uncluttered). Each row has a Mark Recovered button -> markInjuryRecovered API call which flips isActiveInjury=false on the backend.

SettingsScreen.js (HomeStack)

Loads getUserById(userId) into form fields: fullName, email, birthYear, gender, height, weight. Save calls updateUser(userId, {...}). Has trainee/coach toggle (cosmetic only). Privacy + Terms buttons show static Alert text. Bottom 'Connect to Coach / Trainee' navigates to ConnectQR.

ConnectQRScreen.js (HomeStack)

Two modes: 'show' renders a QR code containing JSON {app: 'TrainWise', type: 'coach-connect', userId, timestamp}; 'scan' shows a paste-text area + 'Open Camera' stub. handleConnect validates JSON has app=='TrainWise', shows confirm Alert. No backend call yet - the connect handshake is local-only.

ProfileScreen.js (ProfileTab)

Reads 'user' from useAuth and renders rows: Full Name, Email, Username, Activity Level, Experience Level, Height, Weight, Role (Coach/Trainee). Single Logout button calls useAuth().logout().

GoogleFitScreen.js (HealthTab, lives in src/api/)

Health Connect UI + workout management. On mount: checkHCPPermissions, loadWorkouts. Pull-to-refresh reloads activity logs. Buttons: Request Permissions, Sync Now (calls triggerSync), Manual Refresh. Each row has a per-row Delete button (Alert confirm then deleteActivityLog + recalc). Renders FlatList of synced workouts with last-sync timestamp + error banner when sync fails.

8. End-to-End Data Flows

Login flow (working)

```

LoginScreen handleSubmit
  -> useAuth().login(email, password)
    AuthContext.login
      -> apiLogin(email, password)
        src/api/api.js POST /api/auth/login
        AuthController.Login
          -> UserBL.Authenticate(email, password)
            -> UserDAL.LoginByEmailPassword
              (calls sp_LoginUser)
            <- returns full User row
          persists user object to AsyncStorage,
          exposes userId via useAuth.
  -> isLoggedIn flips true,
  AppNavigator swaps from AuthStack to AppStack.

```

Add workout flow (working)

```

User taps Apply on AddWorkoutScreen
-> services/api.createActivityLog(payload)
    POST /api/activitylog
    ActivityLogController.Create
    -> ActivityLogBL.Create (validate)
    -> ActivityLogDAL.Insert (sp_InsertActivityLog)
-> services/api.calculateDailyLoad(userId)
    POST /api/dailyload/user/{id}/calculate
    DailyLoadController.Calculate
    -> DailyLoadBL.CalculateForDay
    -> DailyLoadDAL.CalculateDailyLoad
        -- pulls 28d activity (sp_GetActivityLogsForLoad)
        -- pulls thresholds (sp_GetUserLoadContext)
        -- computes acute/chronic/AC/stress/level in C#
        -- persists (sp_SaveDailyLoad)
        -- returns tuple
-> navigation.navigate('WorkoutSummary', { summary: {...} })

```

Sync from Health Connect (Android real-build only)

```

GoogleFitScreen 'Sync Now'
-> useSyncWorkouts.triggerSync
    SyncService.syncWorkoutsToBackend(userId, deviceId, 7)
    step 1: initializeHealthConnect
    step 2: checkPermissions
    step 3: getStructuredWorkouts(start, end) from native HC
    step 4: getActivityLogs(userId) [GET /api/ActivityLog/user/{id}]
    step 5: dedup by 60s startTime tolerance
    step 6: for each new workout: POST /api/ActivityLog
    step 7: PUT /api/users/{userId}/devices/{deviceId} {LastSync}

```

9. Build & Run instructions

9.1 Database

- Open SSMS connected to Lirone\SQLEXPRESS (Windows auth)
- Convert SQL/TrainWiseDB_Script.sql to UTF-8 if SSMS complains about UTF-16 BOM, or open with 'Open with Encoding' -> UTF-16 LE
- Execute the script - creates 16 tables, 49 procs, seeds ActivityTypes / InjuryTypes / TrainingGoals / LoadParameters
- Verify: SELECT * FROM sys.objects WHERE type='P' should return 49 rows

9.2 Backend

```

cd TrainWise_GitHub
dotnet restore
dotnet run

# verify
open http://localhost:5249/swagger

```

WARNING: HTTPS redirect breaks phone testing

Comment out `app.UseHttpsRedirection()` in `Program.cs` before pointing the phone at the backend. The phone won't trust the ASP.NET dev cert and the redirect will hang the request.

9.3 Frontend (Expo Go on Android via USB)

```
# in TrainWiseExpo/
npm install
npx expo start --tunnel          # or --localhost when using adb

# in another terminal (USB plugged in)
adb reverse tcp:5249 tcp:5249  # phone localhost:5249 -> PC :5249

# on phone: open Expo Go, scan QR
```

If using WiFi instead of USB

- Phone and PC must be on the SAME WiFi (no AP isolation)
- Open Windows Defender Firewall, allow inbound TCP 5249
- Edit src/api/api.js AND src/services/api.js, replace 127.0.0.1 with PC's LAN IPv4 (e.g. 192.168.1.117)

10. Known Issues & Architectural Gaps

WARNING: No AuthController exists (login is broken)

Frontend POSTs to /api/auth/login. Database has sp_LoginUser. Backend has no AuthController, no AuthBL, no UserDAL.Login* method. Login throws every time. Fix: add AuthController + AuthBL + UserDAL.LoginByEmailPassword (wraps sp_LoginUser).

WARNING: User model is missing half of the SQL columns

BL/Models/User.cs has 10 properties. The SQL Users table has 22 columns including Email, Password, IsCoach, ExperienceLevel, BaseLineDailyLoad, BaseLineWeeklyLoad, HealthDeclaration, ConfirmTerms, IsBaselineEstablished, BaselineEstablishedDate. Anything UserDAL hands back through this model silently drops those columns.

WARNING: ActivityLog model missing 4 fields

AddWorkoutScreen sends exertionLevel, duration, calculatedLoadForSession, isConfirmed. The C# ActivityLog.cs model has none of them. They reach SQL only because sp_InsertActivityLog accepts them as parameters - but the round-trip (Get / Update) cannot read them back into the model.

WARNING: CoachRecommendationsController and UserDevicesController missing [ApiController] / [RoutePrefix("api/")]

Without [Route("api/...")] ASP.NET cannot map HTTP requests to actions. Both controllers are effectively unreachable. Frontend calls to /api/coachrecommendations/user/{id} and /api/userdevice/user/{id} will 404.

WARNING: Two parallel api.js files

src/api/api.js and src/services/api.js both export overlapping axios calls under different names (getActivityLogs vs getActivityLogsByUser, etc). Different baseUrl conventions (/api included vs not). Hooks-rule fixes had to track imports carefully.

WARNING: Severity scale mismatch

InjuryReportScreen renders a 1-10 picker (10 dots). InjuryReportBL.Create throws if Severity > 5. Today the picker silently submits invalid values. Fix one of: clip in BL, change UI to 1-5, or change validator to 1-10.

WARNING: Plain-text 8-char password

Users.Password is char(8), stored as plain text. sp_LoginUser compares with COLLATE Latin1_General_CS_AS (case-sensitive). When auth is wired up, hash before storing.

WARNING: No auth middleware

Program.cs calls UseAuthorization but never AddAuthentication. Every endpoint is public. Adding [Authorize] would have no effect.

WARNING: Hooks at module scope was the source of the red-screen crash

The `useContext of null` red error on the phone was caused by 5 screens calling `const { userId } = useAuth();` at the TOP of the file, outside the component function. React hooks MUST be inside a component body. All five are now fixed - keep the rule in mind for any new screen.

11. File Index

Frontend (TrainWiseExpo/src/)

App.js Provider chain root
 index.js registerRootComponent
 app.json Expo config
 assets/images/wowowow.png Brand logo
 src/api/AuthContext.js Session context + useAuth()
 src/api/api.js axios + login + ActivityLog CRUD
 src/api/HealthConnectService.js Real native HC wrapper
 src/api/HealthConnectService.mock.js Expo Go fallback
 src/api/SyncService.js Health Connect -> backend pipeline
 src/api/useSyncWorkouts.js Hook: triggerSync, state
 src/api/GoogleFitScreen.js Health Connect UI
 src/services/api.js Full CRUD axios surface
 src/navigation/NavigationStack.js AppNavigator + tabs
 src/screens/WelcomeScreen.js Brand cover
 src/screens/LoginScreen.js Email/password login
 src/screens/HomeScreen.js Dashboard
 src/screens/StatsScreen.js Charts + edit
 src/screens/WarningsDashboardScreen.js Load level + AC ratio
 src/screens/AddWorkoutScreen.js Manual workout entry
 src/screens/WorkoutSummaryScreen.js Result card after add
 src/screens/InjuryReportScreen.js Injury logging + drill-down
 src/screens/ActiveInjuriesScreen.js Active list + Mark Recovered
 src/screens/SignUpScreen.js Step 1 of registration
 src/screens/SignUpFinal.js Step 2 of registration
 src/screens/SettingsScreen.js Profile editor

src/screens/ConnectQRScreen.js Coach/trainee handshake
 src/screens/ProfileScreen.js Profile + logout
 src/components/Card.js Rounded card wrapper
 src/components/PrimaryButton.js Pink CTA button
 src/components/ScreenHeader.js Title + back arrow
 src/theme/colors.js Color/Font/Spacing tokens
 src/constants/google.js GOOGLE_WEB_CLIENT_ID

Backend (TrainWise/TrainWise/)

Program.cs ASP.NET startup
 appsettings.json ConnectionString
 TrainWise.csproj Net8 project
 TrainWise.sln Solution
 Controllers/UsersController.cs
 Controllers/ActivityLogController.cs
 Controllers/DailyLoadController.cs
 Controllers/RecommendationController.cs
 Controllers/InjuryReportController.cs
 Controllers/InjuryTypesController.cs
 Controllers/CoachController.cs
 Controllers/CoachRecommendationsController.cs (broken attrs)
 Controllers/TrainingGoalsController.cs
 Controllers/UserGoalsController.cs
 Controllers/UserActivityPreferencesController.cs
 Controllers/UserDevicesController.cs (broken attrs)
 BL/UserBL.cs / DailyLoadBL.cs / ActivityLogBL.cs / etc (12)
 BL/Models/*.cs 15 POCO models
 DAL/DBservice.cs Connect + sproc helper
 DAL/UserDAL.cs / ActivityLogDAL.cs / DailyLoadDAL.cs / etc (14)

Database

../SQL/TrainWiseDB_Script.sql UTF-16LE encoded - convert to UTF-8 before reading with most tools

Documentation

generate_docs_pdf.py THIS script (PyMuPDF)
 TrainWise_Project_Documentation.pdf Output of this script

End of document

Regenerate at any time: py generate_docs_pdf.py